Learning-Augmented KV-Cache Retention for Tool-Using LLM Agents

David Mei Omkar Lashkare Brian Park Zijie Shen Hangting Lu

Stony Brook University

Research presentation · Fall 2026

How should an AI system manage GPU memory while an agent waits for a tool?

What a KV cache is

Every new token has to look back at every earlier one. The **KV cache** stores that work so it is not redone.

1. **Model reads the prompt** builds the cache

↓

2. **Agent calls a tool** runs tests, greps a repo

↓

3. **Agent waits** cache holds memory, generates nothing

↓

4. **Agent resumes** reuse the cache, or rebuild it

A short history

2017	The Transformer. Attention needs every earlier token.
2018–21	Caching those tensors becomes standard practice.
2022	Continuous batching. Many requests share one GPU.
2023	PagedAttention / vLLM. The cache is managed in pages.
2024	InferCept. Tool-call pauses become a scheduling problem.
2025–26	Continuum gives the cache a **time-to-live**.

The field solved *how to store* the cache. **How long to keep it is still open.**

The decision at every pause

The agent may wait one second or ten minutes. We must choose now.

Retain

Instant resume.
Holds memory others want.

Offload

Frees the GPU.
Costs a transfer each way.

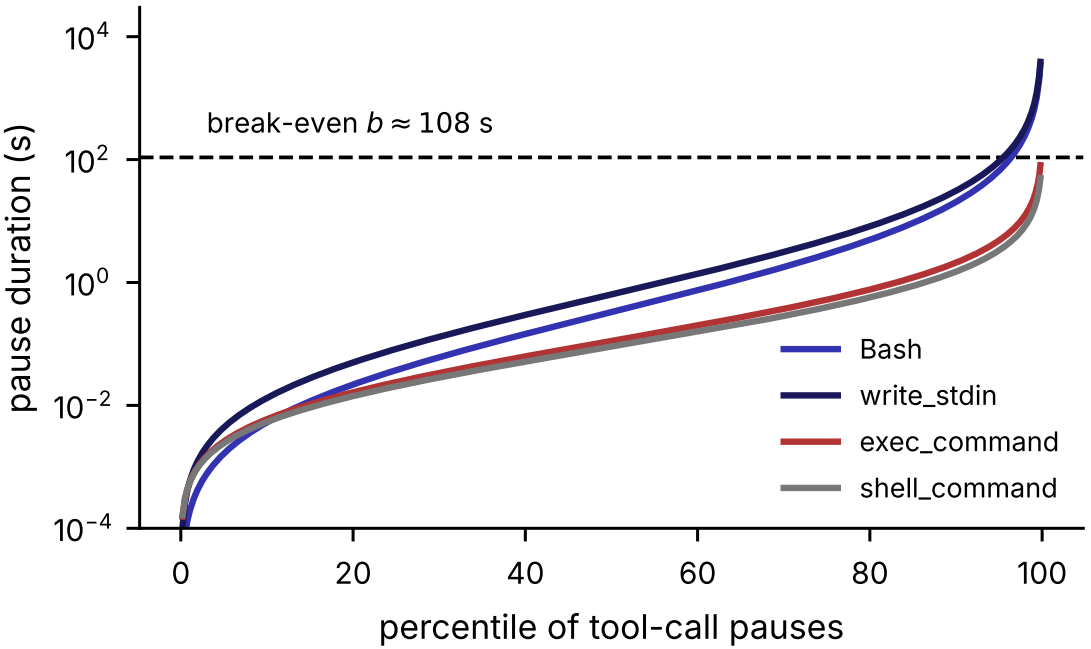
Evict

Frees memory now.
Pays a rebuild if it returns.

Question for you — to start us off

You are the scheduler and you do not know how long the wait is. **Which do you pick?**

Pauses vary enormously



`Bash` pauses:

- median **0.33 s**
- 90th **20 s**
- 99th **300 s**

A thousand-fold spread.

A policy tuned for the typical pause is wrong on the ones that cost most.

Published workload statistics (Continuum, TraceLab). Not our measurements.

The trade-off: this is ski rental

Pay a little every second you keep the cache, or pay once if you drop it and the agent comes back.

Keep it

$c \times$ time held

Drop it, and be wrong

B (one rebuild)

Break-even time

$$b = \frac{B}{c}$$ the hold time that costs exactly one eviction

Idle server $b \approx 2$ s. Busy server $b \approx 100$ s. **It moves with load.**

What people do today

Approach	Strength	Limitation
Evict immediately	frees memory at once	rebuilds on every turn
Fixed TTL	simple, predictable	one number for every tool
Prediction-aware TTL	adapts to the load	only as good as the guess

State of the art: Continuum (ICLR 2026)

A TTL from reload cost and queueing delay, with a duration distribution per tool. Reports **1.1–3.7× lower delay** on Llama-3.1.

The framing is right. **Our question is one term in their cost model.**

Our concern with the cost model

What a pause should cost

$$J(\tau) = c\,\mathbb{E}[\min(T, \tau)] \;+\; B\,\mathbb{P}(T > \tau)$$

T the real pause $\quad$ τ our hold time $\quad$ c memory per second $\quad$ B rebuild

The issue

Rent should stop when the agent *returns*, not when our timer expires. Charging for the whole timeout overstates the cost, and evicts **too early**.

Question for you — a design choice

A **fixed timeout**, or a policy that **estimates whether the agent is coming back**? What would make you trust the second?

Our proposal: use the prediction, inside a fence

1. Start from the break-even time $b = B/c$
2. Allow only a safe range around it

$$W_\lambda = [\,\lambda b,\ b/\lambda\,]$$

3. Let the prediction pick inside it

The dial λ

Small: wide fence, more trust.
$\lambda = 1$: no trust, hold exactly b.

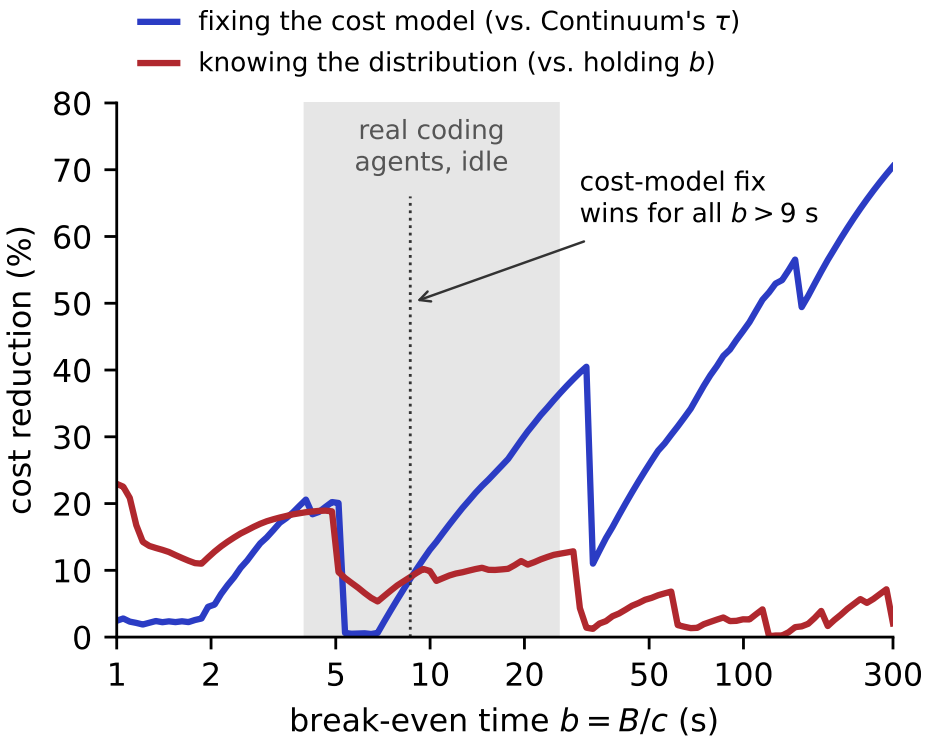
How we will test it

Four policies on identical traces: evict now, fixed TTL, hold b, ours. Measure latency, recomputed tokens and memory, sweeping prediction error from accurate to badly wrong.

Keep what a good prediction buys. Bound what a bad one costs.

Proposed, not built. No experiments run yet.

When is prediction actually worth it?



The literature disagrees:

- **Continuum**: predicting pays.
  tool calls, seconds
- **Choi & Joshi**: it is unnecessary.
  human approvals, minutes to hours

Our reading: **both hold, for different kinds of pauses**. Load is a second factor.

In TraceLab, tool pauses take 0.18 s and human waits about 2 min (medians).

Real agents sit at $b \approx 4$–26 s even when idle: **fix the cost model first.**

One-pause model on 314,589 measured TraceLab tool pauses. Not a measured system.

Pros, cons, and the alternatives

What it gives you

Skips a rebuild on resume. No model change, no accuracy loss. Pure scheduling.

What it costs

Holds memory others want. The benefit depends on load. A predictor is one more thing to be wrong.

Combines with

Quantised caches and grouped-query attention shrink B. Offload tiers make eviction cheaper.

Alternatives

Always evict. Offload to host memory. Token-level eviction (H2O, SnapKV). Or buy more memory.

We are not replacing any of these. **We are deciding when to stop holding on.**

Where it is used, and is it open source?

In industry

vLLM is incubating at the **LF AI & Data Foundation**, maintained with UC Berkeley, AWS, IBM, Databricks and others. **Eight of ten** official Llama 3.1 inference partners serve with it.

Open source

Framework	License	GitHub stars
vLLM	Apache-2.0	∼94,000
SGLang	Apache-2.0	∼35,500

Continuum has a preview branch.

Our repository

A **research plan** and a measured pause table (351,497 pauses). No policy implementation yet.

Next: longer sessions, human approvals measured in minutes, and multi-agent systems sharing one pool.

Published results exist for Continuum and Choi & Joshi. Ours are planned.

Opportunities: careers, and the money

This stack is a job requirement, today

Together AI's *Inference Optimization Engineer* listing asks for **vLLM or SGLang** and for **"KV cache systems: PagedAttention or similar"**. That is this project, in a job description.

Where the spending is going

Gartner, Aug 2026: AI infrastructure-as-a-service reaches **$42B in 2026**, and for the first time **inference ($23.3B) outspends training ($19B)**.

Training is a one-off. **Inference recurs on every request**, and agents make many.

Opportunities: you already pay for this

The same decision, with a price on it

Anthropic's prompt caching charges **1.25**× to write a cache that lives 5 minutes, **2**× for one hour, and **0.1**× to read it.

Choosing 5 minutes or 1 hour **is** ski rental, and you are the one choosing.

And if you run models locally

On a consumer GPU it is the **KV cache, not the weights**, that usually caps how long a context you can hold.

Question for you — for your own projects

Where else do you **throw something away, then rebuild it moments later**?

Takeaways, and what to read

Three things to take away

1. Agents pause constantly, and their cache keeps holding GPU memory.
2. How long to hold is a **ski-rental** problem.
3. Whether *predicting* helps appears to depend on **server load**. That is what we will test.

Recommended reading

Continuum, arXiv:2511.02230 — the paper we are arguing with.
Purohit, Svitkina & Kumar, NeurIPS 2018 — ski rental with predictions.
TraceLab, arXiv:2606.30560 — the agent traces.

`github.com/Omkar38/agent-kv-retention`